

Benchmarking, Docker, and Load Testing

Server Engineering Labs

Author:

Laura Ortiz Moreno

Topics Covered:

- System benchmarking
- ApacheBench load testing
- Docker containers
- Docker Compose orchestration
- JMeter testing
- Infrastructure monitoring

Environment:

Debian Linux Server
Rocky Linux Client VM
Virtualized lab environment

Contents

1	Introduction	2
2	Benchmarking Fundamentals	2
3	Installing Phoronix Test Suite	2
3.1	Monitoring During Benchmarking	3
4	HTTP Load Testing with ApacheBench	3
5	Docker Fundamentals	4
5.1	Running Containers	5
6	Docker Compose and Microservices	5
6.1	Monitoring Resource Usage	6
7	Infrastructure Monitoring During Load	6
8	Load Testing with Apache JMeter	7
8.1	API Authentication Testing	7
8.2	Monitoring Container Performance	8
8.3	Performance Summary	9
9	Performance Analysis	9
10	Challenges Encountered	9
11	Conclusion	10

1. Introduction

This report documents several benchmarking and infrastructure performance experiments performed in a Linux virtualized environment.

The laboratory focused on:

- System benchmarking with Phoronix Test Suite
- HTTP load testing with ApacheBench
- Containerized deployments using Docker
- Multi-container orchestration with Docker Compose
- Stress testing using Apache JMeter
- Resource monitoring during workload execution

The exercises combined benchmarking tools, monitoring systems, and containerized services to simulate realistic server workloads.

2. Benchmarking Fundamentals

Two main categories of benchmarking were discussed:

- System benchmarking
- Service benchmarking

The importance of using external clients to perform measurements was emphasized in order to avoid affecting the server under test.

3. Installing Phoronix Test Suite

Phoronix Test Suite was installed on the Debian server.

Installation:

```
sudo apt install phoronix-test-suite
```

Additional dependencies:

```
sudo apt install curl unzip
```

Benchmark execution example:

```
phoronix-test-suite benchmark pts/sudokut
```

```
Phoronix Test Suite v10.8.4

To Install: pts/apache-3.0.0

Determining File Requirements .....
Searching Download Caches .....

1 Test To Install
  5 Files To Download [21.08MB]
  208MB Of Disk Space Is Needed
  2 Minutes, 29 Seconds Estimated Install Time

pts/apache-3.0.0:
  Test Installation 1 of 1
  5 Files Needed [21.08 MB]
  Downloading: httpd-2.4.56.tar.bz2 [7.11MB]
  Downloading .....
  Downloading: http-test-files-1.tar.xz [2.24MB]
  Estimated Download Time: 1m .....
  Downloading: apr-1.7.2.tar.bz2 [0.85MB]
  Estimated Download Time: 1m .....
  Downloading: apr-util-1.6.3.tar.bz2 [0.41MB]
  Estimated Download Time: 1m .....
  Downloading: wrk-4.2.0.tar.gz [10.47MB]
  Estimated Download Time: 1m .....
  Approximate Install Size: 208 MB
  Estimated Install Time: 2 Minutes, 29 Seconds
```

Figure 1: Installation and initialization of the Phoronix Test Suite benchmarking framework.

3.1. Monitoring During Benchmarking

Resource utilization was monitored during benchmark execution using:

- CPU usage
- Memory utilization
- Service responsiveness

4. HTTP Load Testing with ApacheBench

ApacheBench (ab) was used to generate concurrent HTTP requests against the server.

Example:

```
ab -n 5000 -c 50 http://server-address/
```

Parameters:

- -n: total number of requests
- -c: concurrency level

The tests provided:

- Request throughput
- Latency statistics
- Error rates
- Response time percentiles

```

Benchmarking 10.10.10.105 (be patient).....done

Server Software:      Apache/2.4.65
Server Hostname:      10.10.10.105
Server Port:          80

Document Path:        /
Document Length:       10703 bytes

Concurrency Level:     5
Time taken for tests:  0.113 seconds
Complete requests:     50
Failed requests:        0
Total transferred:     548850 bytes
HTML transferred:      535150 bytes
Requests per second:   442.35 [#/sec] (mean)
Time per request:      11.303 [ms] (mean)
Time per request:      2.261 [ms] (mean, across all concurrent requests)
Transfer rate:         4741.86 [kbytes/sec] received

Connection Times (ms)
      min    mean[+/-sd] median    max
Connect:    0       2   1.5      1       7
Processing:  1       9   7.0      7      29
Waiting:    1       4   4.6      2      23
Total:      2      10   8.1      8      35

Percentage of the requests served within a certain time (ms)
 50%    8
 66%   10
 75%   10
 80%   14
 90%   29
 95%   30
 98%   35
 99%   35
100%   35 (longest request)

```

Figure 2: ApacheBench performance results showing concurrent request handling and throughput metrics.

5. Docker Fundamentals

The laboratory introduced containerization concepts using Docker.

Key concepts included:

- Images
- Containers
- Isolation
- Lightweight virtualization

Docker was installed:

```
sudo apt install docker.io
```

Verification:

```
sudo docker ps
```

5.1. Running Containers

Example container execution:

```
sudo docker run hello-world
```

Unused containers and images were cleaned with:

```
sudo docker system prune -a -f
```

6. Docker Compose and Microservices

Docker Compose was used to orchestrate multiple services simultaneously.

The laboratory included:

- Application containers
- MongoDB containers
- Service orchestration
- Resource monitoring

Repository cloning example:

```
git clone <repository>  
cd application-directory
```

Container startup:

```
sudo docker compose up -d
```

```
root@debian:/opt# cd ISE-P4App/  
root@debian:/opt/ISE-P4App# cat docker-compose.yml  
version: '2.0'  
services:  
  #MongoDB based in the original Mongo Image  
  mongodb:  
    image: mongo  
    ports:  
      - "27017:27017"  
  
  # Initialize mongodb with data  
  mongodinit:  
    build: ./mongodb  
    links:  
      - mongodb  
  
  # Nodejs App  
  nodejs:  
    build: ./nodejs  
    ports:  
      - "3000:3000"  
    links:  
      - mongodb
```

Figure 3: Docker Compose configuration used to orchestrate multi-container application services.

6.1. Monitoring Resource Usage

Resource utilization was monitored during deployment:

- CPU usage
- Memory consumption
- Container statistics

Useful command:

```
docker stats
```

7. Infrastructure Monitoring During Load

System monitoring was performed during benchmarking and stress tests using Zabbix dashboards.

The monitoring infrastructure allowed visualization of:

- CPU spikes
- Memory usage
- Network activity
- System load during stress tests

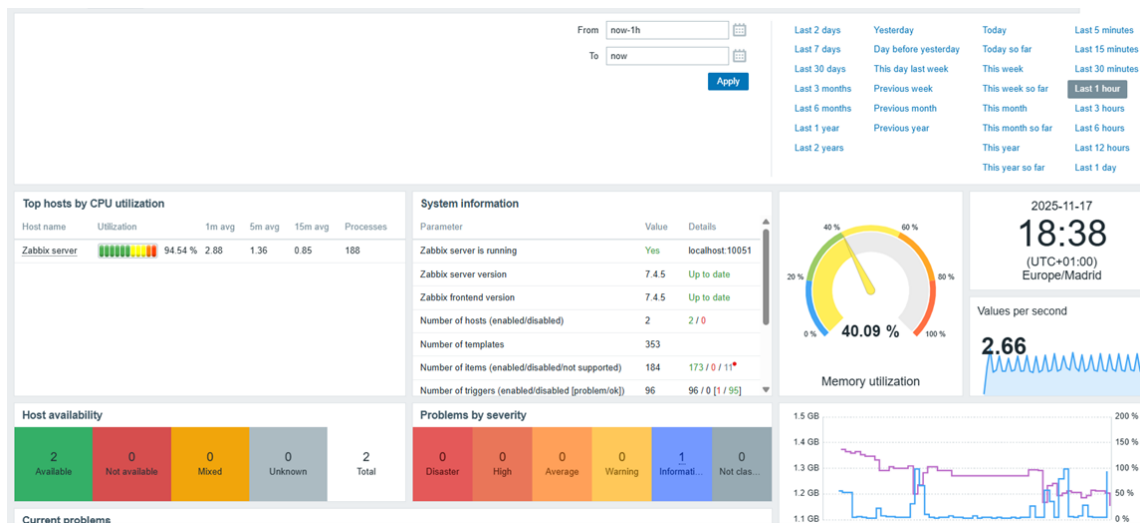


Figure 4: Zabbix dashboard displaying high resource utilization during intensive benchmarking operations.

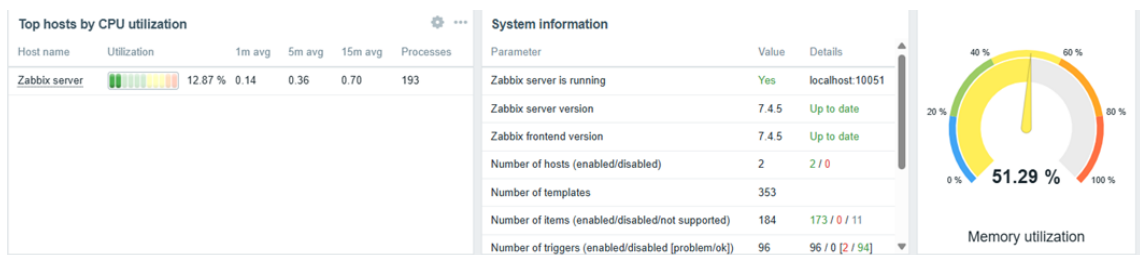


Figure 5: System stabilization after load testing with normalized CPU and memory consumption metrics.

8. Load Testing with Apache JMeter

Apache JMeter was used to simulate concurrent users and API requests.

The tests included:

- Login requests
- Authenticated API requests
- Concurrent user simulation
- Performance statistics

JMeter execution example:

```
jmeter -n -t test-plan.jmx -l results.jtl
```

Parameters:

- -n: non-GUI mode
- -t: test plan file
- -l: output results file

```
root@debian:/opt/ISE-P4App# sudo docker compose up -d
WARN[0000] /opt/ISE-P4App/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 3/3
  ✓ Container ise-p4app-mongodb-1      Started      0.0s
  ✓ Container ise-p4app-nodejs-1       Started      0.0s
  ✓ Container ise-p4app-mongodbininit-1 Start...     0.0s
root@debian:/opt/ISE-P4App# docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS
PORTS         NAMES
d79ba53f162e   ise-p4app-mongodbininit  "docker-entrypoint.s..." 6 days ago    Up 30 second
27017/tcp     ise-p4app-mongodbininit-1
67067d7f76f3   ise-p4app-nodejs       "docker-entrypoint.s..." 6 days ago    Up 29 second
0.0.0.0:3000->3000/tcp, :::3000->3000/tcp   ise-p4app-nodejs-1
7dc563f7ad91   mongo                 "docker-entrypoint.s..." 6 days ago    Up 35 second
0.0.0.0:27017->27017/tcp, :::27017->27017/tcp   ise-p4app-mongodb-1
```

Figure 6: Containerized application stack successfully deployed using Docker Compose prior to load testing.

8.1. API Authentication Testing

The laboratory also explored:

- Basic authentication
- Token extraction
- Authorization validation
- Role-based access testing

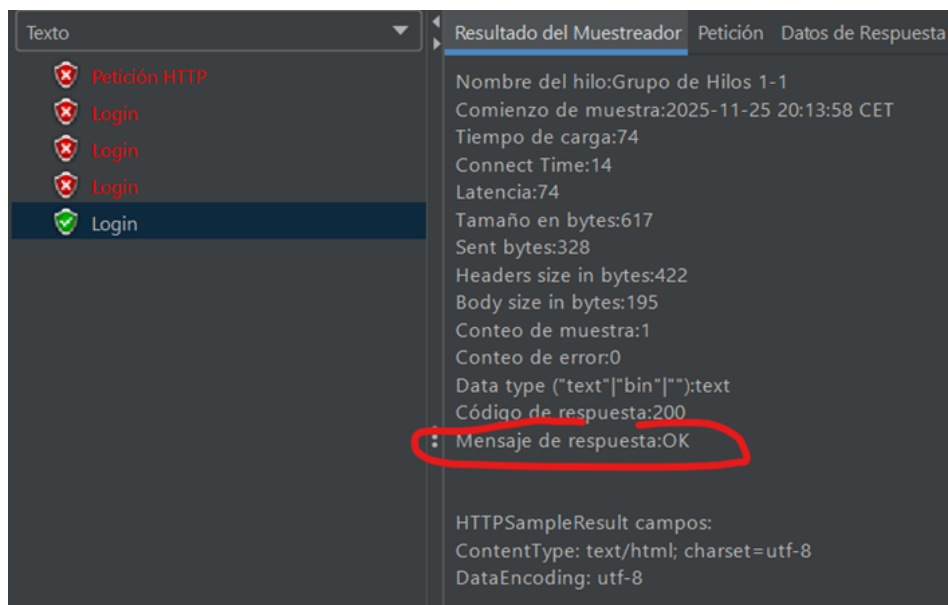


Figure 7: Successful authentication request executed through Apache JMeter.

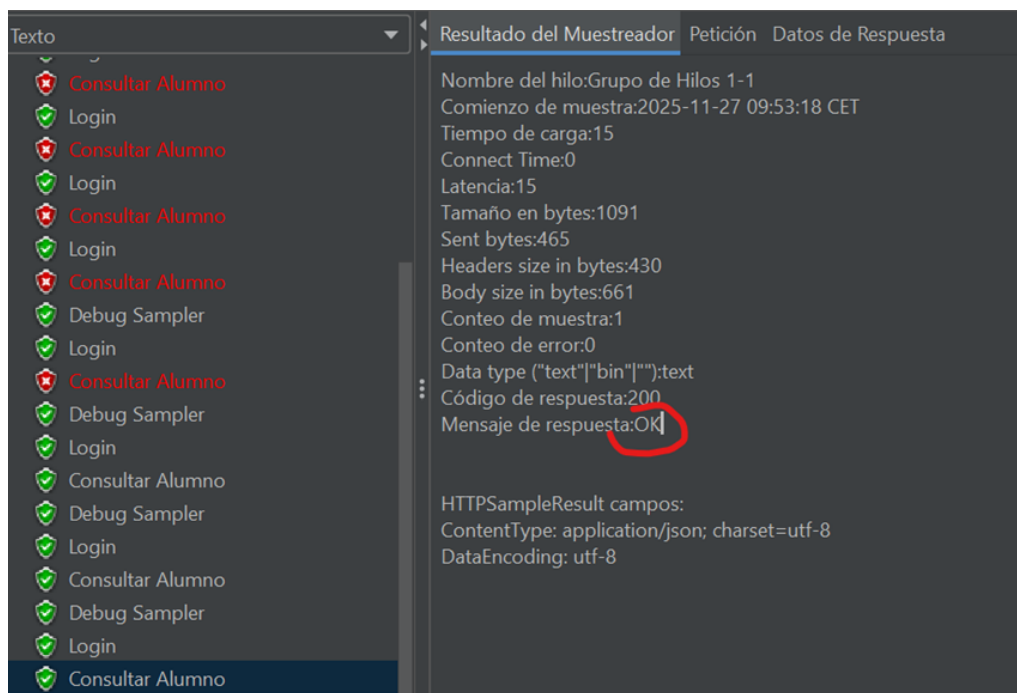


Figure 8: API request validation performed using authenticated JMeter test scenarios.

8.2. Monitoring Container Performance

Resource usage during stress testing was monitored directly from Docker.

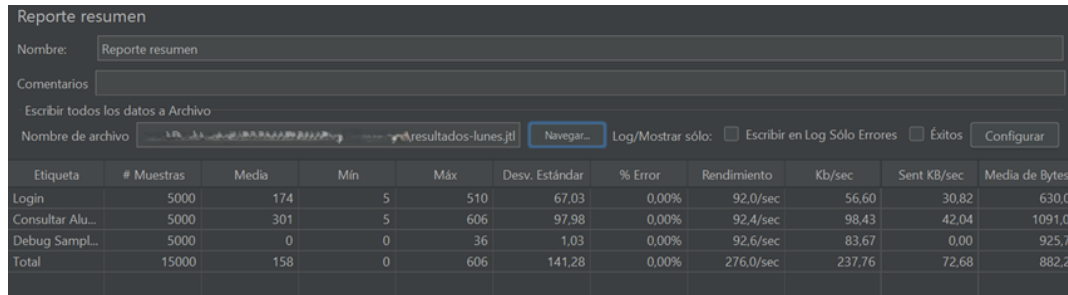
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
67067d7f76f3	ise-p4app-nodejs-1	0.00%	96.46MiB / 1.928GiB	4.89%	1.13MB / 449kB	219MB / 0B	23
7dc563f7ad91	ise-p4app-mongodb-1	3.99%	574.6MiB / 1.928GiB	29.11%	1.58MB / 1.39MB	1.04GB / 127MB	68

Figure 9: Real-time container resource monitoring using `docker stats` during concurrent load testing.

8.3. Performance Summary

JMeter generated aggregated statistics including:

- Throughput
- Average response time
- Error percentage
- Request latency



Etiqueta	# Muestras	Media	Mín	Máx	Desv. Estándar	% Error	Rendimiento	Kb/sec	Sent KB/sec	Media de Bytes
Login	5000	174	5	510	67,03	0,00%	92,0/sec	56,60	30,82	630,0
Consultar Alu...	5000	301	5	606	97,98	0,00%	92,4/sec	98,43	42,04	1091,0
Debug Sampl...	5000	0	0	36	1,03	0,00%	92,6/sec	83,67	0,00	925,7
Total	15000	158	0	606	141,28	0,00%	276,0/sec	237,76	72,68	882,2

Figure 10: Apache JMeter summary report showing aggregated performance metrics under concurrent load conditions.

9. Performance Analysis

Performance metrics analyzed included:

- Throughput
- Average response time
- Error percentage
- Resource utilization
- Container performance

The monitoring environment allowed observation of CPU and memory behavior under stress conditions.

10. Challenges Encountered

Several issues appeared during the laboratory:

- Missing package dependencies
- Container startup failures
- Resource limitations in virtual machines
- Incorrect API authentication configuration
- High concurrency instability

Troubleshooting these problems provided practical experience in infrastructure performance testing.

11. Conclusion

This laboratory introduced practical benchmarking and load-testing workflows in Linux infrastructure environments.

The exercises demonstrated:

- System benchmarking techniques
- Service stress testing
- Containerized deployments
- Multi-service orchestration
- Monitoring under load
- API performance validation

These concepts are highly relevant in DevOps, backend infrastructure, performance engineering, and cloud-native environments.